

Answer the questions in the boxes provided on the question sheets. If you run out of room for an answer, add a page to the end of the document.

Name: Xuming Huang Wisc id: 908 772 9324

Asymptotic Analysis

1. Kleinberg, Jon. *Algorithm Design* (p. 67, q. 3, 4). Take the following list of functions and arrange them in ascending order of growth rate. That is, if function $g(n)$ immediately follows function $f(n)$ in your list, then it should be the case that $f(n)$ is $O(g(n))$.

- (a) $f_1(n) = n^{2.5}$
 $f_2(n) = \sqrt{2n}$
 $f_3(n) = n + 10$
 $f_4(n) = 10n$
 $f_5(n) = 100n$
 $f_6(n) = n^2 \log n$

Solution:

$f_2, f_3, f_4, f_1, f_6, f_5$

- (b) $g_1(n) = 2^{\log n}$
 $g_2(n) = 2^n$
 $g_3(n) = n(\log n)$
 $g_4(n) = n^{4/3}$
 $g_5(n) = n^{\log n}$
 $g_6(n) = 2^{(2^n)}$
 $g_7(n) = 2^{(n^2)}$

Solution:

$g_3, g_4, g_5, g_1, g_2, g_7, g_6$

2. Kleinberg, Jon. *Algorithm Design* (p. 68, q. 5). Assume you have a non-decreasing function f such that $f(n) \geq 1$ and a non-decreasing function g such that $g(n) \geq 2$. Also, assume that $f(n)$ is $O(g(n))$. For each of the following statements, decide whether you think it is true or false and give a proof or counterexample.

(a) $f(n)^2$ is $O(g(n)^2)$

Solution: True

$$f(n) = O(g(n)) = \{f(n) : \exists c, n_0 > 0 \mid 0 \leq f(n) \leq c \cdot g(n) \ \forall n > n_0\}$$

$$\text{thus } 0 \leq f(n_0)^2 \leq c^2 \cdot g(n_0)^2$$

Therefore, there exists $c' = c^2$ makes $0 \leq f(n)^2 \leq c' g(n)^2 \ \forall n > n_0$

(b) $2^{f(n)}$ is $O(2^{g(n)})$

Solution: False.

$$f(n) = n$$

$$g(n) = 2n$$

(c) $\log_2 f(n)$ is $O(\log_2 g(n))$

Solution:

$$0 \leq \log_2 f(n) \leq \log_2 c g(n) = \log_2 c + \log_2 g(n)$$

$$\text{when } c > 1, \ 0 \leq \log_2 f(n) \leq (\log_2 c + 1) \log_2 g(n)$$

$$\text{thus } \log_2 f(n) = O(\log_2 g(n))$$

3. Kleinberg, Jon. *Algorithm Design* (p. 68, q. 6). You're given an array A consisting of n integers. You'd like to output a two-dimensional n -by- n array B in which $B[i, j]$ (for $i < j$) contains the sum of array entries $A[i]$ through $A[j]$ — that is, the sum $A[i] + A[i + 1] + \dots + A[j]$. (Whenever $i \geq j$, it doesn't matter what is output for $B[i, j]$.) Here's a simple algorithm to solve this problem.

```

for i = 1 to n
  for j = i + 1 to n
    add up array entries A[i] through A[j]
    store the result in B[i, j]
  endfor
endfor

```

- (a) For some function f that you should choose, give a bound of the form $O(f(n))$ on the running time of this algorithm on an input of size n (i.e., a bound on the number of operations performed by the algorithm).

Solution:

$$O(n^3)$$

- (b) For this same function f , show that the running time of the algorithm on an input of size n is also $\Omega(f(n))$. (This shows an asymptotically tight bound of $\Theta(f(n))$ on the running time.)

Solution:

Look at some of the operation

$$\text{let } i \leq \frac{n}{4}, j \geq \frac{3n}{4},$$

then $j - i \geq \frac{3n}{4} - \frac{n}{4} = \frac{n}{2}$ thus at least $\frac{n}{2}$ will be added up

Totally, there're $\frac{1}{4}n$ choice for i and j , thus

$$f(n) = \frac{1}{4}n \cdot \frac{1}{4}n \cdot \frac{1}{2}n = \left(\frac{n^3}{32} \right) \Rightarrow \Omega(f(n)) = \Omega(n^3)$$

- (c) Although the algorithm provided is the most natural way to solve the problem, it contains some highly unnecessary sources of inefficiency. Give a different algorithm to solve this problem, with an asymptotically better running time. In other words, you should design an algorithm with running time $O(g(n))$, where $\lim_{n \rightarrow \infty} \frac{g(n)}{f(n)} = 0$.

Solution:

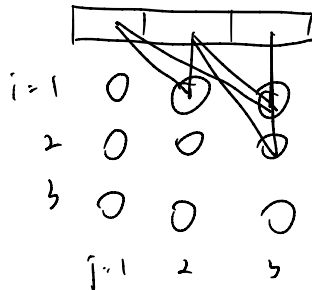
$result = A[i]$

```

for i = 1 to n ✓
  for j = i + 1 to n
    add up array entries A[i] through A[j]
    store the result in B[i, j]
  endfor
endfor

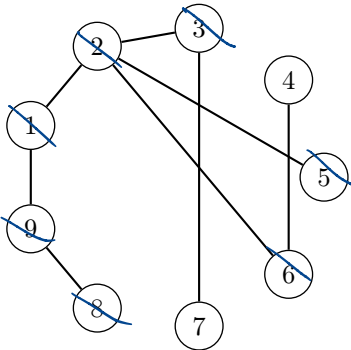
```

Instead, 'add A[j] to result'



Graphs

4. Given the following graph, list a possible order of traversal of nodes by breadth-first search and by depth-first search. Consider node 1 to be the starting node.



Solution:

1, 2, 9, 3, 5, 6, 8, 7, 4

5. Kleinberg, Jon. *Algorithm Design* (p. 108, q. 5). A binary tree is a rooted tree in which each node has at most two children. Show by induction that in any binary tree the number of nodes with two children is exactly one less than the number of leaves.

Solution:

$$N(n) = N(L) - 1$$

Base Case: Root $1 = 2 - 1 = 1$



Induction Step:

Assume $N(n) = j - 1$ $1 \leq j \leq k$ satisfies $N(L) = j$ so that

$$N(n) = k - 1 = k - 1 = N(L) - 1$$

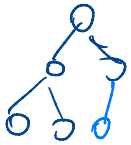
When adding new child,

①. only child, $N(n)$ and $N(L)$ remain

②. second child, $N(n) = k - 1 + 1$

$$N(L) = k + 1$$

$$\Rightarrow N(n) = k + 1 - 1 = N(L) - 1$$



6. Kleinberg, Jon. *Algorithm Design* (p. 108, q. 7). Some friends of yours work on wireless networks, and they're currently studying the properties of a network of n mobile devices. As the devices move around, they define a graph at any point in time as follows:

There is a node representing each of the n devices, and there is an edge between device i and device j if the physical locations of i and j are no more than 500 meters apart. (If so, we say that i and j are "in range" of each other.)

They'd like it to be the case that the network of devices is connected at all times, and so they've constrained the motion of the devices to satisfy the following property: at all times, each device i is within 500 meters of at least $\frac{n}{2}$ of the other devices. (We'll assume n is an even number.) What they'd like to know is: Does this property by itself guarantee that the network will remain connected?

Here's a concrete way to formulate the question as a claim about graphs:

Claim: Let G be a graph on n nodes, where n is an even number. If every node of G has degree at least $\frac{n}{2}$, then G is connected.

Decide whether you think the claim is true or false, and give a proof of either the claim or its negation.

Solution: True.

Inductive.

Base: Q_0 $\text{Deg}(N) = 1 = \frac{1}{2} \cdot N(G)$

Induct. $N(G) = j$ $2 \leq j \leq k$ holds

Assume $N(G) = k$ and $\text{deg}(n_i) \geq \frac{k}{2}$ then G is connected

When $N(G) = k+2$ and $\text{deg}(n_i) \geq \frac{k}{2} + 1$

We know k nodes are connected, and the additional 2 nodes needs to be proved they are connected to previous graph.

By contradiction, We assume 2 additional node can be isolated then $\text{deg}(n_{\text{add}}) \leq 1$ (connect to each other). However,

$\text{deg}(n_i) \geq \frac{k}{2} + 1 \geq 2$ so contradiction.

Thus, 2 nodes are connected to previous graph

Therefore, $N(G) = k+2$ and $\text{deg}(n_i) \geq \frac{k}{2} + 1$, then G connected

Coding Question: DFS

7. Implement depth-first search to visit all connected components in either C, C++, C#, Java, Python, or Rust. Given an undirected graph with n nodes and m edges, your code should run in $O(n + m)$ time. Remember to submit a makefile along with your code, just as with the first coding question.

Input: the first line contains an integer t , indicating the number of instances that follows. For each instance, the first line contains an integer n , indicating the number of nodes in the graph. Each of the following n lines contains several space-separated strings, where the first string s represents the name of a node, and the following strings represent the names of nodes that are adjacent to node s .

The order of the nodes in the adjacency list is important, as it will be used as the tie-breaker. For example, consider an instance

```
4
xy v0 b
b xy
v0 xy a
a v0
```

The tie break priority is $xy \rightarrow v0 \rightarrow b \rightarrow a$, so your search should start at **xy**, then choose **v0** over **b** as the second node to visit. Overall, your code should produce the following output:

```
xy v0 a b
```

Input constraints:

- $1 \leq t \leq 1000$
- $1 \leq n \leq 100$
- Strings only contain alphanumeric characters
- String length is bounded by a constant.
- Strings are guaranteed to be the names of the nodes in the graph.

Output: for each instance, print the names of nodes visited in depth-first traversal of the graph, *with ties between nodes visiting the first node in input order*. Start your traversal with the first node in input order. The names of nodes should be space-separated, the output of each instance should be terminated by a newline, and the lines should have **no trailing spaces**.

Sample Input:

```
8 9
9 1 8
2
3
A B
B A
C
9
1 2 9
2 1 6 5 3
4 6
6 2 4
5 2
3 2 7
7 3
```

Sample Output:

A	B	C
1	2	6
4	5	3
7	9	8

The sample input has two instances. The first instance corresponds to the graph below on the left. The second instance corresponds to the graph below on the right.

